Google Colaboratory

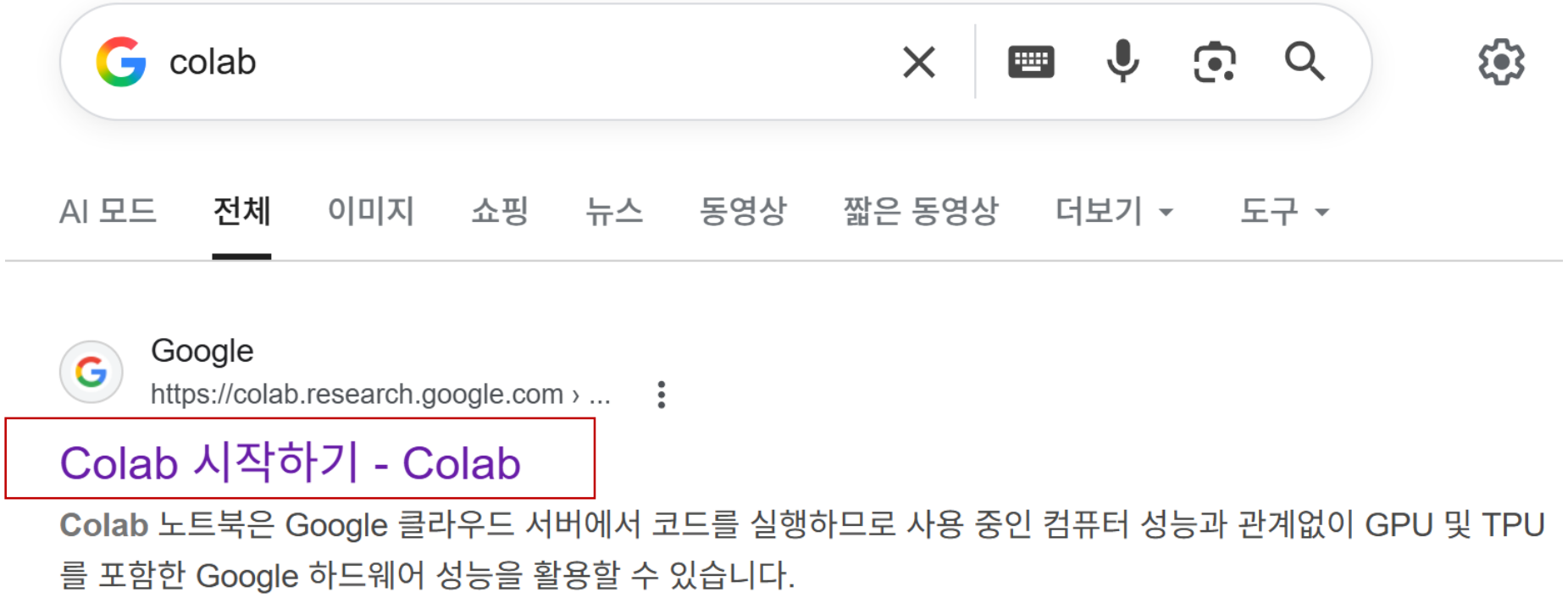
Colab

김창훈박사

Colab 시작!

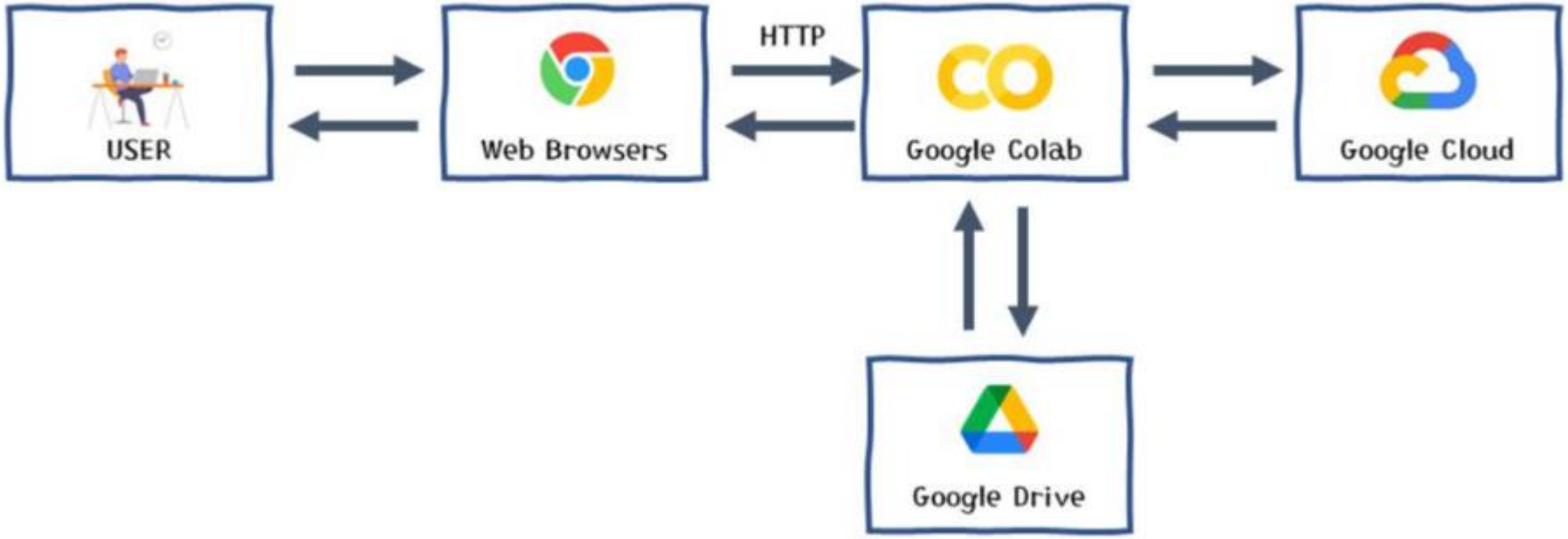
- Cloud 기반 Jupyter Notebook 개발환경
- Colab을 통하여 사용자가 GPU, TPU를 사용
- 사람이 개입하는 머신러닝, AI가 스스로 판단하는 딥러닝 등에 사용
- Google Colaboratory 줄임말

- 구글에서 colab 검색하고, 'Colab 시작하기' 를 선택!



Colab 사용은 인터넷을 기반으로 한다. 구글 계정이 필요.

- 사용자는 구글 계정이 필요!



- GPU
- TPU 등을 지원!

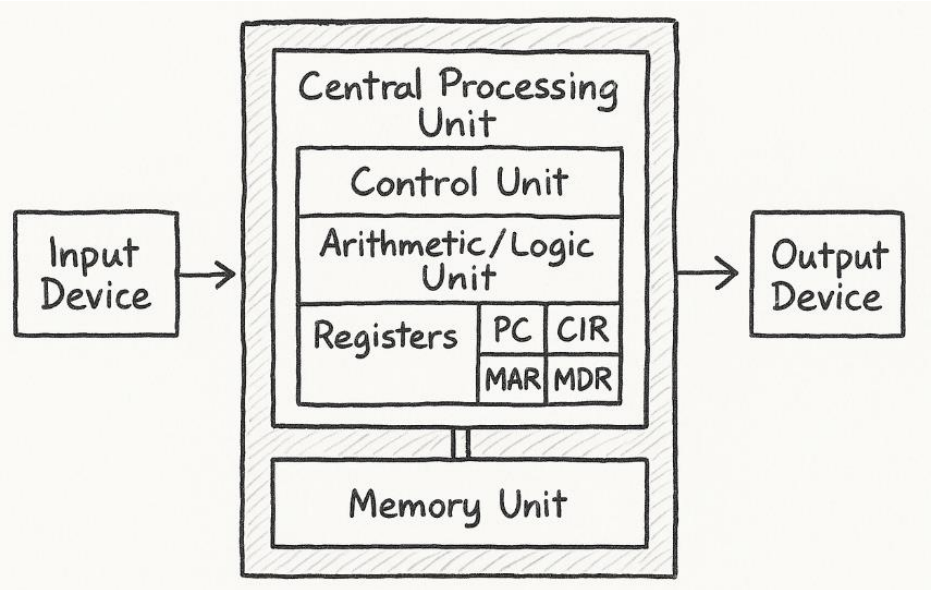
참조: https://wscode.tistory.com/30

컴퓨터 구조

김창훈박사

컴퓨터 구조 : Von Neumann 아키텍처

현대 컴퓨터의 기본 토대는 1945년 수학자 존 폰 노이만(John von Neumann)이 제안한 "폰 노이만 아키텍처"에 기반하고 있습니다. 이 구조의 가장 큰 특징은 '프로그램 내장 방식'입니다. 프로그램(명령어)과 데이터가 모두 같은 메모리에 저장되며, CPU가 이를 순차적으로 읽어 처리합니다. 이 단순하면서도 강력한 모델은 현대 컴퓨터의 표준이 되었습니다.



레지스터는 일종의 메모리이다.

- 명령어 레지스터 IR
- 프로그램 카운터 PC
  : 현재 명령어 CIR와 다음 명령어 주소
- 누산기 레지스터 AC
  : 데이터 저장
- 데이터 레지스터 DR
- 메모리 주소 레지스터 MAR
- 프로그램 레지스터 PC
- 메모리 데이터 레지스터 MDR

폰 노이만 모델에 따르면 컴퓨터는 다음과 같은 기본 구성요소로 나뉩니다:

- 입력장치 (Input Device): 외부 데이터를 컴퓨터 내부로 입력하는 장치입니다.
- 기억장치 (Memory): 명령어와 데이터를 저장하는 장치입니다.
- 처리장치 (CPU): 명령어를 해석하고 실행하는 중앙 처리 장치입니다.
- 출력장치 (Output Device): 처리 결과를 사용자에게 전달하는 장치입니다.
- 제어장치 (Control Unit): 컴퓨터 전체 시스템의 동작을 조율하는 역할을 합니다.

https://brunch.co.kr/@daehyub/74

컴퓨터 Computer 구조

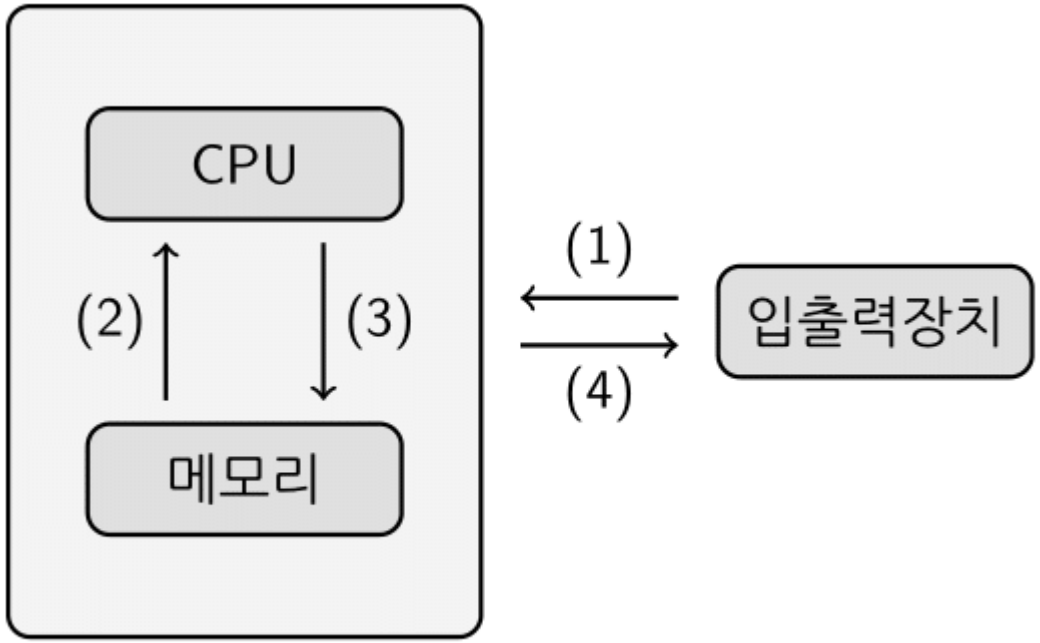


그림: 폰 노이만 구조

(1). CPU는 메모리에서 명령을 순서대로 읽어옵니다. 명령들은 메모리의 어떤 값을 불러와야 하는지, 값을 어떻게 계산해야 하는지, 어떤 값을 메모리에 다시 저장해야 하는지 등 CPU가 해야 하는 모든 일을 명시해 줍니다

(2). 계산이 끝난 이후 메모리에 저장된 값은 (3) 다시 입출력 장치를 통해 출력됩니다 (4)

참고: https://stementor.tistory.com/entry

중앙처리장치 CPU 는 0 과 1 로 이루어진 신호를 인식해 받아 들인다.

레지스터(Register) : CPU가 연산을 수행하기 위해 사용하는 임시 저장 장치(임시 저장소)

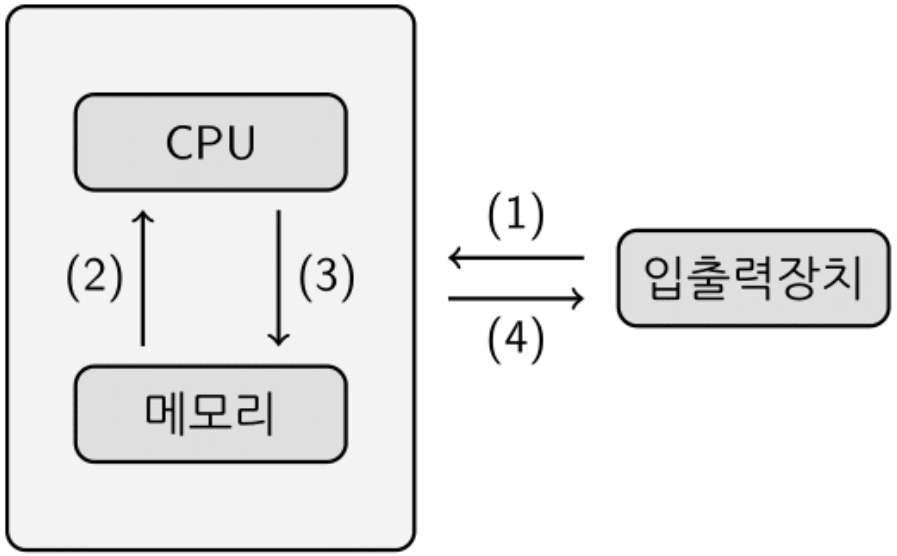


그림: 폰 노이만 구조

CPU는 메모리의 특정 위치에 저장되어 있는 **명령**(instruction)들을 순서대로 불러와 처리합니다.

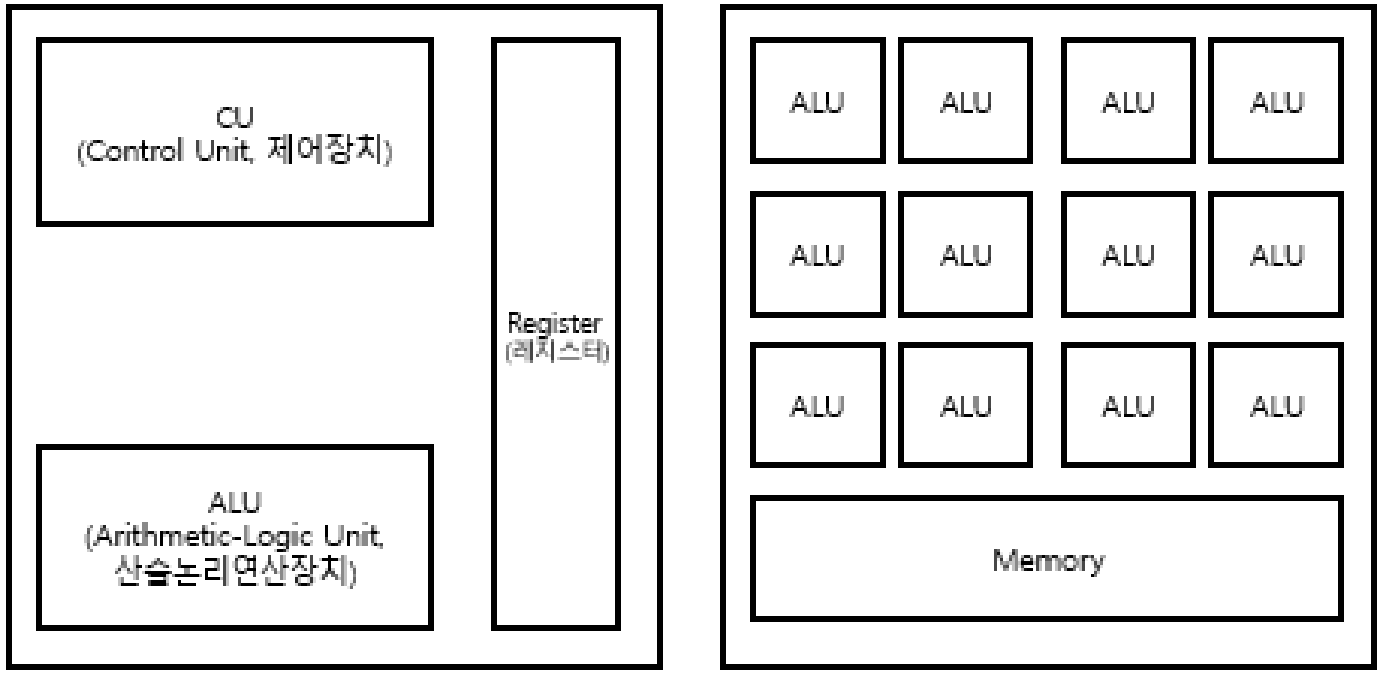
CPU가 처리하는 명령의 몇 가지 예시를 살펴봅시다.

- 두 레지스터에 저장된 값을 더해 다른 레지스터에 저장한다.
- 한 레지스터가 메모리 상의 주소를 가리키고 있을 때, 이 주소에 저장된 값을 다른 레지스터로 불러온다.
- 한 레지스터가 메모리 상의 주소를 가리키고 있을 때, 다른 레지스터에 저장된 값을 이 주소에 저장한다.
- 한 레지스터에 적힌 값이 1이라면 다음 차례에 바로 다음 명령을 실행하지 말고, 명시된 다른 명령을 실행한다.

참고: https://stementor.tistory.com/entry

CPU 와 GPU 구조

CPU (좌) 와 GPU (우) 의 구조



참고: https://wooono.tistory.com/325

ALU 와 CPU

ALU(산술논리연산장치)란?

- **산술연산을 진행하는 장치입니다.**
- 딥러닝 알고리즘은 본질적으로, 많은 양의 **단순 사칙연산(행렬 곱셈 등)**을 수행합니다.
- **단순 사칙연산은 병렬화가 가능하기 때문에,**
- **ALU 가 많다면, 단순 사칙연산을 병렬화함으로써, 딥러닝을 보다 효율적으로 최적화 할 수 있습니다.**

CPU

- **'Central Processing Unit'의 약자이며, 컴퓨터의 모든 연산과 작업을 처리하는 장치입니다.**
- **CPU 의 ALU 는 1개입니다.**
  - CPU 에는 제어장치(CU)가 존재하는 대신, ALU 가 1개 뿐입니다.
  - **따라서, 단순 사칙연산 속도가, GPU 에 비해 안 좋습니다.**
  - 하지만, 복잡한 연산에는 용이합니다.

GPU 와 TPU

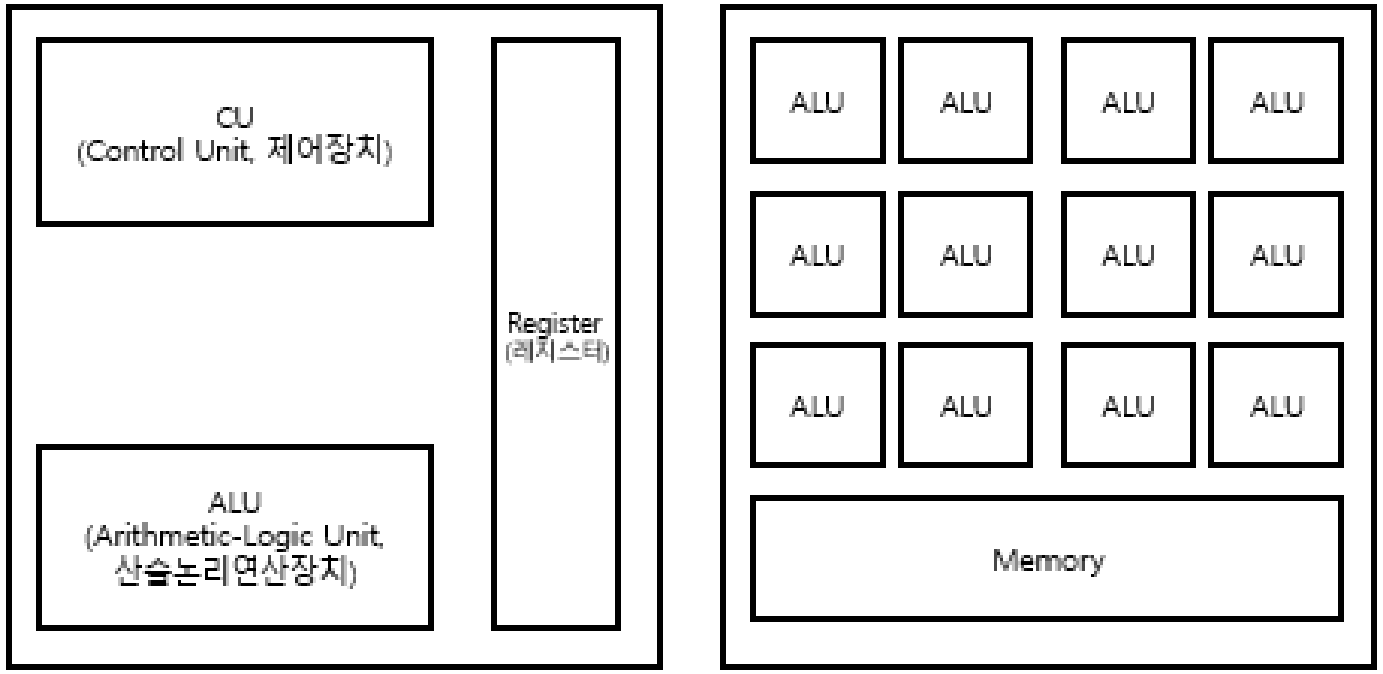
GPU

- 이전에는 그래픽 카드의 핵심부품으로 알려져있었으며,
- 3D 그래픽의 다양한 효과(입체감, 광원 등)를 CPU 혼자서는 처리하기 힘들기 때문에, 그래픽 연산 처리를 위해 개발되었습니다.
- **하지만, 현재는 딥 러닝 영역에서 컴퓨팅 워크로드를 가속화하기 위해, 보다 광범위하게 활용되고 있습니다.**
- **GPU 의 ALU 는 약 2500개입니다.**
  - **ALU 의 개수가 CPU 보다 많기 때문에, CPU 에 비해 단순 사칙연산 속도가 빠릅니다.**

TPU

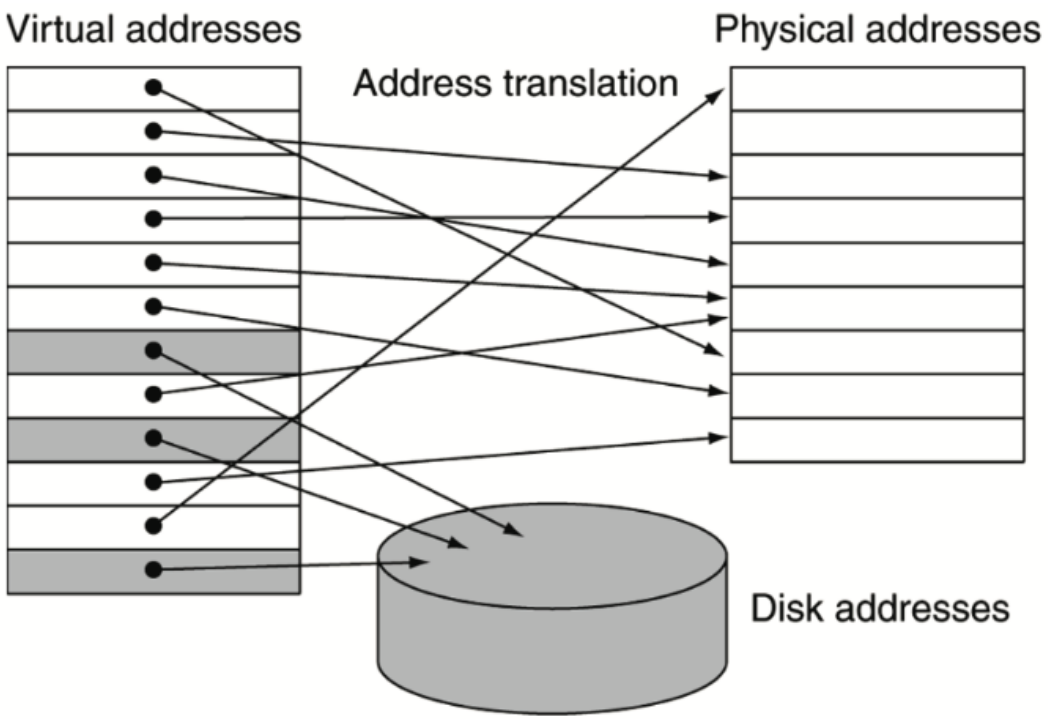
- **'Tensor Processing Unit' 의 약자이며, Google 이 자체적으로 개발한 칩입니다.**
- **Google 의 오픈 소스 머신 러닝 프레임워크인 TensorFlow 에 최적화 되어있는, 맞춤형 집적 회로입니다.**
- **TPU 의 ALU 는 약 3만개입니다.**
  - **TPU 는 벡터 / 행렬연산에 용이하도록, ALU 의 위치별 용도가 미리 설계되어있습니다.**
  - **따라서, TPU 는 GPU 에 비해 특정 환경에서 훨씬 많은 연산력을 자랑합니다.**

CPU (좌) 와 GPU (우) 의 구조



Memory 메모리

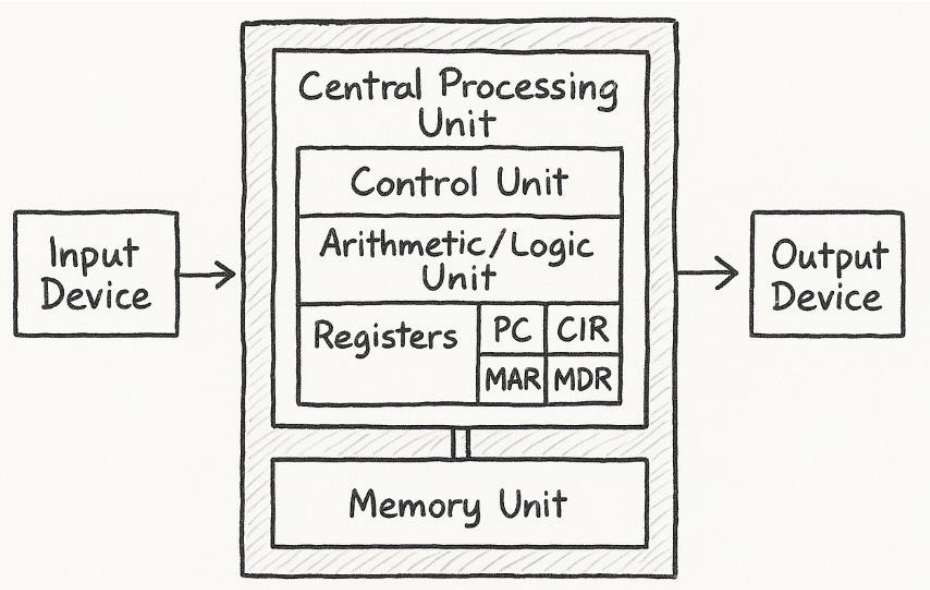
- 프로그램은 메모리 상에서 돌아간다.
- 메모리에는 주소가 각각 배정되어 있고, 각 주소 마다 값을 저장한다.
- CPU가 요청한 주소에 적힌 값을 CPU에게 알려 주거나,
  반대로 CPU가 알려 준 주소에 새로운 값을 저장하는 역할을 수행한다.
- 여러 프로그램이 동시에 수행되면, 메모리 배정과 갱신에 혼란이 발생한다.



가상 메모리(Virtual Memory) 에서 **주소 변환**(address translation) 또는 **주소 사상**(address mapping) 과정

컴퓨터 구조 : Von Neumann 아키텍처

현대 컴퓨터의 기본 토대는 1945년 수학자 존 폰 노이만(John von Neumann)이 제안한 "폰 노이만 아키텍처"에 기반하고 있습니다. 이 구조의 가장 큰 특징은 '프로그램 내장 방식'입니다. 프로그램(명령어)과 데이터가 모두 같은 메모리에 저장되며, CPU가 이를 순차적으로 읽어 처리합니다. 이 단순하면서도 강력한 모델은 현대 컴퓨터의 표준이 되었습니다.



폰 노이만 모델에 따르면 컴퓨터는 다음과 같은 기본 구성요소로 나뉩니다:

- 입력장치 (Input Device): 외부 데이터를 컴퓨터 내부로 입력하는 장치입니다.
- 기억장치 (Memory): 명령어와 데이터를 저장하는 장치입니다.
- 처리장치 (CPU): 명령어를 해석하고 실행하는 중앙 처리 장치입니다.
- 출력장치 (Output Device): 처리 결과를 사용자에게 전달하는 장치입니다.
- 제어장치 (Control Unit): 컴퓨터 전체 시스템의 동작을 조율하는 역할을 합니다.

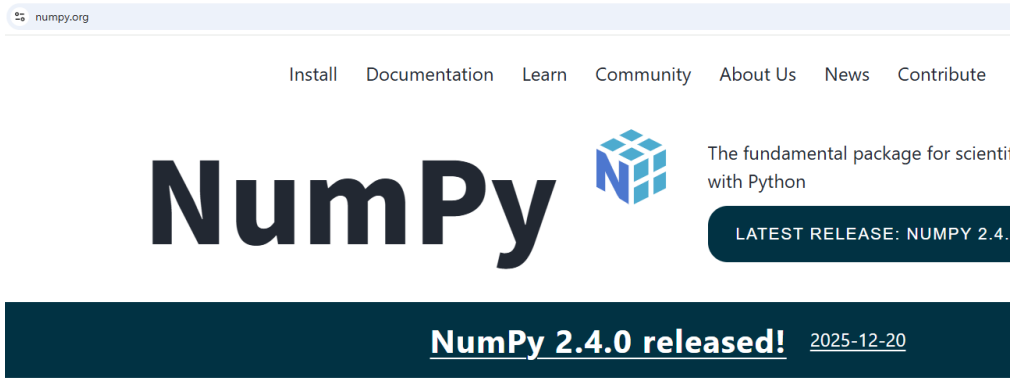
Numpy

김창훈박사

NumPy 는 Python 라이브러리 이다.

"NumPy(Numerical Python)는 파이썬에서
대규모 다차원 배열과 행렬 연산을 고속으로 처리하기 위한
핵심 과학 계산 라이브러리입니다.

- Numpy는 1995년 공개되었다.
- Numpy는 C로 개발된 소프트웨어
- N-dimensional arrays 계산에 뛰어남
- Numerical computing tools : Linear Algebra routines, Fourier Transforms, …



QuTiP : Quantum Toolbox in Python

그냥 참고!

https://qutip.org/

QuTiP는 개방형 양자 시스템의 동역학을 시뮬레이션하는 오픈 소스 소프트웨어입니다.

QuTiP is open-source software for simulating the dynamics of open quantum systems.

QuTiP 라이브러리는 뛰어난 NumPy, Scipy, Cython 수치 계산 패키지에 의존합니다. 또한 Matplotlib을 사용하여 그래프 출력을 제공합니다.

QuTiP는 양자 광학, 이온 포획, 초전도 회로, 양자 나노기계 공진기 등 다양한 물리 응용 분야에서 흔히 발견되는 임의의 시간 의존성을 갖는 해밀토니안을 포함하여 다양한 해밀토니안에 대한 사용자 친화적이고 효율적인 수치 시뮬레이션을 제공하는 것을 목표로 합니다.

https://youtu.be/DUK5mHvy0Bw

GitHub

https://github.com/changkim88/stat